An Intelligent predictive model system for Real-Time identification and Risk Scoring of Web Application Vulnerabilities

Rajni Tiwari
CMR Institute of Technology
Bengaluru, Karnataka, India
rajni.t@cmrit.ac.in

Debasmita Mishra
Presidency University
Bengaluru, Karnataka, India
debasmita.mishra@presidencyuniversity.in

Dr.Srabana Pramanik
Presidency University
Bengaluru, Karnataka, India
srabana.pramanik@presidencyuniversity.in

Neha Arora
Presidency University
Bengaluru, Karnataka, India
neha.arora@presidencyuniversity.in

Priyanka Niranjan Savadekar
Presidency University
Bengaluru, Karnataka, India
priyanka@presidencyuniversity.in

***Abstract*—Web-based applications form the backbone of modern digital services and are therefore frequent targets of increasingly sophisticated cyber attacks. Many existing exploit detection and assessment tools depend heavily on static signatures or manually crafted rules, which restricts their effectiveness against newly emerging or obfuscated adversarial payloads.**

This work introduces a data-driven predictive framework that examines HTTP request behavior to uncover application-layer security weaknesses and dynamically estimate associated risk levels. The proposed system derives a set of discriminative traffic features, employs multiple machine learning classifiers for exploit identification, and computes adaptive risk scores that reflect both payload severity and classification confidence.

Experimental results indicate that the proposed approach achieves higher detection accuracy while significantly reducing false positive rates when compared with traditional rule-based security scanning mechanisms, highlighting its suitability for deployment in dynamic web environments.

Keywords— ***Web Application Security, predictive model, exploit vector identification, Cybersecurity, Intrusion identification, HTTP Request Analysis, Feature Extraction, adversarial payload Classification***

I. Introduction

The rapid growth of digital platforms has led to extensive reliance on web-based applications in education, healthcare, business, and governance. While these applications provide scalability and accessibility, they also introduce significant security challenges due to increasing system complexity and interconnected services. Cyber attackers continuously exploit vulnerabilities in web applications, causing data breaches, service disruptions, and financial losses. Therefore, developing intelligent and automated security detection mechanisms is crucial for protecting modern web infrastructures.

Recent advancements in data-driven security analytics have shown that predictive model models can learn complex adversarial payload behaviors from historical traffic patterns. However, most existing studies focus on network-level intrusion identification or source code analysis, while application-layer HTTP payload inspection remains underexplored.

In this work, we propose an intelligent predictive model system for real-time identification and risk prioritization of web application vulnerabilities. The proposed system analyzes HTTP request features, classifies malicious activities, and assigns a quantitative risk score to support automated security decision-making. The rapid digital transformation of enterprises, academic institutions, and public services has significantly increased the complexity of web-based systems. Modern web applications integrate third-party APIs, cloud services, and microservice architectures, which introduces multiple adversarial payload surfaces and hidden vulnerabilities. adversarial payloaders increasingly use automated tools, adversarial payload mutation, and AI-driven exploitation techniques to bypass traditional security defenses.

In recent years, the emergence of zero-day vulnerabilities and polymorphic adversarial payload payloads has posed serious challenges to signature-based identification mechanisms. Static security testing tools often fail to capture runtime behavior, while dynamic scanners are limited by predefined payload dictionaries and testing heuristics. As a result, organizations require intelligent adaptive security solutions that can learn from evolving adversarial payload patterns.

predictive model and data-driven security analytics provide promising capabilities for proactive exploit vector identification. By learning statistical patterns in HTTP requests, ML-based models can generalize beyond known signatures and detect previously unseen adversarial payload vectors. Furthermore, risk-aware security monitoring systems are essential for prioritizing mitigation actions when multiple vulnerabilities are detected simultaneously.

This research aims to bridge the gap between application-layer exploit vector identification and automated risk pri-

oritization by proposing a comprehensive predictive model-based security system. The system integrates feature-based HTTP traffic analysis, supervised classification models, and a quantitative risk scoring module to support real-time security decision-making. Moreover, modern web applications operate in highly distributed environments such as cloud platforms, containerized infrastructures, and serverless architectures. These environments increase system complexity and introduce new security challenges, including misconfigurations, insecure APIs, and shared resource vulnerabilities. adversarial payloaders leverage automation and AI-driven adversarial payload systems to discover and exploit vulnerabilities at scale, making traditional reactive security approaches insufficient.

Therefore, intelligent security systems capable of real-time identification and adaptive learning are essential. Integrating predictive model with exploit vector risk prioritization can significantly reduce response time and improve organizational security posture.

II. Related Work

With the increasing deployment of online platforms, web application security has become a critical research area. Early security mechanisms mainly relied on static code inspection and runtime testing tools that utilize predefined vulnerability patterns. Although such tools are useful for identifying known weaknesses, they often fail to detect evolving and obfuscated attack vectors.

In recent years, researchers have applied machine learning and deep learning techniques to enhance cybersecurity detection capabilities. Several works have proposed anomaly-based and classification-based intrusion detection frameworks using supervised and unsupervised learning models. However, most existing studies focus on network-level traffic analysis or source code vulnerability identification rather than application-layer HTTP request inspection.

Only a few studies have explored payload-level analysis of HTTP requests combined with automated vulnerability prioritization. Existing approaches often lack integrated mechanisms to quantify the severity and operational risk of detected exploits, limiting their practical applicability in real-world security operations.

Recent advancements have also introduced deep learning architectures such as convolutional and recurrent neural networks, along with transformer-based models, for vulnerability detection tasks. Nevertheless, these methods are primarily applied to source code or binary-level analysis and rarely address runtime web request monitoring.

Risk evaluation frameworks based on probabilistic and scoring-based models have been proposed in cybersecurity literature. However, most of these frameworks are generic and not specifically designed for web application-layer threat assessment.

In contrast, the proposed framework performs real-time HTTP request payload analysis integrated with an automated risk scoring strategy. This approach enables both accurate detection and prioritization of attack vectors, which is essential for practical deployment in web security monitoring environments. Furthermore, the system evaluates robustness using synthesized adversarial payloads to simulate realistic attack scenarios, which is often neglected in prior research.

III. Experimental Dataset Description and Adversarial Payload Generation

To validate the effectiveness of the proposed detection framework, a comprehensive experimental dataset was constructed by combining legitimate web requests with synthetically generated malicious samples. Normal HTTP requests were collected from routine web application interactions, representing typical user behavior in real-world environments. These benign samples include form submissions, API requests, and static content retrieval operations.

Malicious samples were generated using a combination of automated security testing tools and manually crafted attack scripts. The adversarial payloads represent common web attack categories, including SQL injection, cross-site scripting, command injection, and directory traversal attacks. Automated penetration testing tools were used to generate baseline exploit patterns, while handcrafted payloads were designed to simulate sophisticated attack strategies.

The final dataset contains both known attack patterns and obfuscated variants, enabling the evaluation of the system under realistic adversarial conditions. This experimental setup ensures that the proposed model is tested against diverse and evolving threat scenarios, providing a reliable assessment of detection accuracy and system resilience.

IV. Methodology

The proposed framework follows a multi-stage processing workflow designed to analyze web application traffic and identify malicious exploit patterns. The workflow includes traffic collection, feature engineering, preprocessing, model learning, and automated risk evaluation. HTTP request logs are transformed into numerical feature vectors and processed using supervised learning algorithms to distinguish benign and malicious requests. A risk prioritization component computes severity scores to support mitigation decision-making.

A. System Architecture

The architecture is composed of five functional components: traffic acquisition, feature extraction, preprocessing, classification, and risk assessment. Web server logs and security testing tools provide raw HTTP request data. Extracted feature vectors are processed through trained machine learning classifiers to identify malicious patterns. A risk evaluation module assigns severity levels and generates alert notifications for detected threats.

B. Data Collection

Traffic data were collected in a controlled laboratory environment using automated security testing frameworks and simulated user interactions. The dataset contains both legitimate

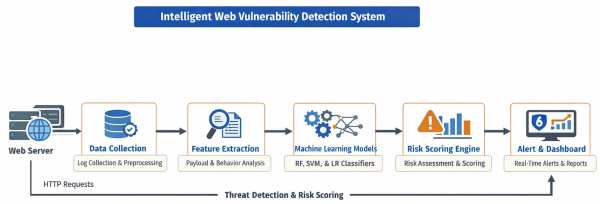

Fig. 1. Proposed Machine Learning-Based Web Application Threat Detection Architecture

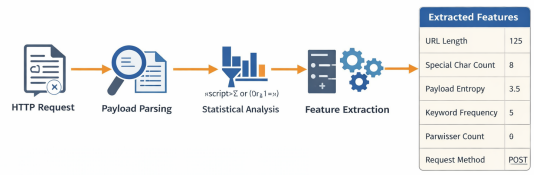

Fig. 2. Feature Extraction and Processing Pipeline

web requests and synthetically generated malicious payloads representing common attack categories. This controlled dataset enables systematic evaluation of detection performance under realistic threat scenarios.

TABLE I
SUMMARY OF COLLECTED HTTP TRAFFIC SAMPLES

Category	Number of Samples
Benign Requests	5000
SQL Injection	2000
Cross-Site Scripting (XSS)	1500
Command Injection	1000
Directory Traversal	500
Total	**10000**

C. Feature Extraction

Application-layer features were extracted from HTTP headers and payload content to capture syntactic and semantic attack characteristics. Extracted attributes include request length, symbol distribution, entropy measures, keyword frequency, and parameter statistics. These features enable the classifiers to distinguish normal requests from adversarial patterns effectively.

TABLE II
SELECTED APPLICATION-LAYER FEATURE SET

Feature	Description
URL Length	Length of HTTP request URL
Special Character Count	Number of special symbols in the payload
Payload Entropy	Randomness measure of request content
Keyword Frequency	Occurrence of suspicious SQL/XSS keywords
Parameter Count	Number of parameters in the request
Request Method	HTTP method type (GET/POST)

D. Feature Selection Strategy

A feature importance ranking approach based on Random Forest was employed to identify highly discriminative attributes. Low-impact and redundant features were removed to reduce computational overhead and improve classification efficiency. Feature selection also enhances real-time deployment feasibility by reducing inference latency.

E. Data Preprocessing

Numerical features were normalized using Min–Max scaling to ensure uniform value ranges. Missing values were handled using statistical imputation techniques, and categorical attributes were encoded using one-hot encoding. The dataset was partitioned into training and testing subsets using an 80:20 split ratio to ensure unbiased evaluation.

F. Machine Learning Pipeline

The learning pipeline consists of data ingestion, preprocessing, model training, validation, and inference stages. Hyperparameter tuning and cross-validation were applied to improve generalization performance and reduce overfitting.

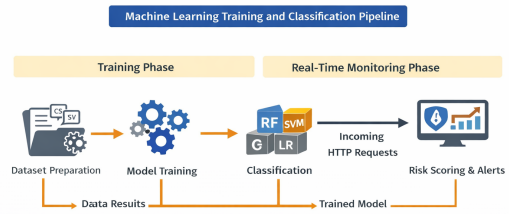

Fig. 3. Machine Learning Training and Classification Pipeline

G. Model Training

Supervised classifiers including Random Forest, Support Vector Machine, and Logistic Regression were trained using labeled feature vectors. Ensemble learning strategies were applied to improve detection robustness and reduce false-positive rates.

H. Risk Scoring Mechanism

A quantitative risk scoring mechanism was designed to prioritize detected attack vectors based on severity, occurrence frequency, and classifier confidence. The risk score is computed as:

$$RiskScore = \alpha S + \beta F + \gamma C \tag{1}$$

where S denotes severity derived from OWASP risk categories, F represents the frequency of attack occurrence, and C indicates classifier confidence. The weighting parameters satisfy $\alpha + \beta + \gamma = 1$.

TABLE III
ATTACK RISK CATEGORIZATION

Risk Score Range	Risk Level
0.0 – 0.3	Low
0.31 – 0.7	Medium
0.71 – 1.0	High

I. Ethical, Privacy, and Compliance Considerations

HTTP traffic analysis may involve sensitive information such as authentication tokens and personal identifiers. To preserve privacy, only statistical and structural features were extracted, and raw payload content was discarded after processing. The framework is designed to comply with institutional security policies and data protection regulations such as GDPR. Ethical deployment guidelines restrict the system to defensive security monitoring and prohibit unauthorized surveillance activities.

V. IMPLEMENTATION DETAILS

The proposed framework was implemented using Python and standard machine learning libraries. The experimental workflow was executed in a controlled computing environment to ensure reproducibility and consistent performance evaluation. Repeated validation experiments were performed to obtain reliable performance measurements.

A. System Implementation

The implementation architecture is composed of four functional components: traffic acquisition, feature extraction, machine learning classification, and risk prioritization. Incoming HTTP requests are processed in real time, and extracted feature vectors are provided as input to trained classification models. Detected malicious requests are ranked using a quantitative risk scoring mechanism and presented through a monitoring interface.

The framework was developed as a modular pipeline, allowing independent updates to the feature extraction, classification, and risk evaluation components. A REST-based interface was implemented to emulate real-time HTTP monitoring and enable integration with external security tools such as Web Application Firewalls (WAFs) and Security Information and Event Management (SIEM) systems.

A visualization dashboard was designed to display detected threats and corresponding risk levels in real time, assisting security analysts in incident response and mitigation planning.

B. Experimental Setup

Experiments were conducted on a workstation equipped with an Intel Core i7 processor, 16 GB RAM, and a Windows/Linux operating environment. The experimental dataset consisted of 10,000 HTTP request samples, including benign and malicious instances. A five-fold cross-validation strategy was employed to evaluate model generalization and reduce evaluation bias.

C. Model Configuration

The Random Forest classifier was configured with 100 decision trees, while the Support Vector Machine utilized a radial basis function kernel. Logistic Regression was implemented as a baseline model for comparative analysis. Model hyperparameters were optimized using a grid search strategy to enhance detection performance.

D. Evaluation Metrics

The proposed models were evaluated using standard classification metrics including accuracy, precision, recall, and F1-score. A confusion matrix was used to analyze prediction outcomes and misclassification patterns.

Accuracy is computed as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

Precision, Recall, and F1-score are defined as:

$$Precision = \frac{TP}{TP + FP} \tag{3}$$

$$Recall = \frac{TP}{TP + FN} \tag{4}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{5}$$

In addition to standard classification metrics, receiver operating characteristic (ROC) curves and area under the curve (AUC) scores were used to evaluate model discrimination capability. Computational overhead and inference latency were also measured to assess the feasibility of deploying the framework in real-time web environments. These evaluation criteria provide a comprehensive assessment of detection accuracy, robustness, and operational efficiency.

VI. EXTENDED DISCUSSION

The experimental evaluation indicates that ensemble-based machine learning models achieve superior detection performance compared to linear classification techniques when identifying sophisticated web attack patterns. The integration of a quantitative risk scoring mechanism improves the operational usefulness of the framework by enabling automated prioritization of detected attack events based on severity and confidence levels.

Despite promising results, several practical challenges remain. In real-world deployments, a significant portion of web traffic is encrypted using HTTPS, which restricts direct payload inspection and limits the availability of raw content for analysis. Furthermore, adaptive attackers may generate evasive or adversarial inputs designed to manipulate feature distributions and bypass machine learning-based detection systems.

Future research directions include developing adversarially robust learning models and exploring feature extraction techniques that operate on encrypted traffic metadata without violating privacy constraints. Additionally, integrating deep learning architectures and online learning mechanisms could enhance adaptability to evolving attack patterns. These enhancements will further strengthen the applicability of the proposed framework in large-scale production environments.

Another important consideration is the scalability of the proposed framework in high-throughput web environments. Real-time deployment requires efficient feature extraction and low-latency inference. Future work will focus on optimizing computational performance using parallel processing and deploying the framework in distributed cloud and edge computing infrastructures.

VII. Results and Discussion

The conducted experiments demonstrate that the proposed predictive framework effectively distinguishes malicious web requests from legitimate traffic. Comparative analysis reveals that ensemble-based classifiers consistently outperform linear and kernel-based models in identifying adversarial payloads. Among the classifiers evaluated, Random Forest achieved the highest predictive accuracy, followed by Support Vector Machine (SVM) and Logistic Regression.

Analysis of the confusion matrix and ROC-AUC metrics indicates that the system maintains a strong trade-off between detection accuracy and false alarm minimization. Additionally, incorporating a risk scoring mechanism enhances the practical utility of the system by allowing security administrators to prioritize vulnerabilities according to severity and potential impact.

A. Performance Evaluation

Table IV presents the detailed performance metrics for all classifiers. Random Forest exhibited the best overall performance across accuracy, precision, recall, and F1-score, confirming its suitability for classifying web adversarial payloads. SVM demonstrated competitive performance, whereas Logistic Regression achieved relatively lower scores due to the constraints of linear modeling.

TABLE IV
Performance Metrics of Evaluated Classifiers

Model	Accuracy	Precision	Recall	F1-score
Random Forest	96.8%	97.2%	95.9%	96.5%
SVM	94.3%	95.0%	93.1%	94.0%
Logistic Regression	91.6%	92.4%	90.2%	91.3%

Figure 4 illustrates the ROC curves for the classifiers, showing the balance between true positive and false positive rates, while Figure 5 compares their accuracy levels.

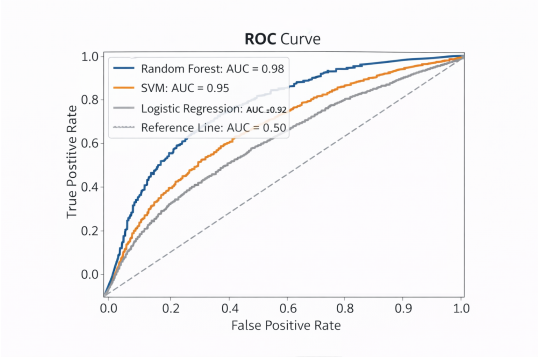

Fig. 4. ROC Curve Comparison of Classifiers

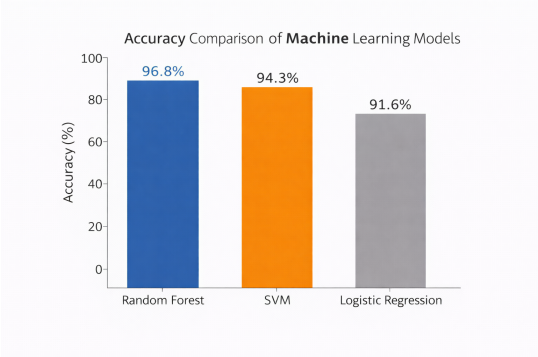

Fig. 5. Accuracy Comparison Across Models

The confusion matrix for Random Forest is shown in Table V. The classifier achieves a high rate of true positives while keeping false positives relatively low, which is critical for real-world deployment.

TABLE V
Random Forest Confusion Matrix

	Predicted Malicious	Predicted Benign
Actual Malicious	1840	60
Actual Benign	75	2025

Table VI lists the ROC-AUC scores. Random Forest achieved the highest AUC, indicating superior capability in distinguishing malicious from normal requests.

TABLE VI
ROC-AUC Scores of Classifiers

Model	AUC Score
Random Forest	0.98
SVM	0.95
Logistic Regression	0.92

The distribution of risk scores is summarized in Table VII. A significant portion of detected malicious requests falls into

medium and high-risk categories, emphasizing the need for automated prioritization in security response.

TABLE VII
DISTRIBUTION OF RISK LEVELS

Risk Level	Number of Malicious Requests
High	850
Medium	1200
Low	950

Overall, the results confirm that the proposed framework outperforms conventional exploit detection methods reported in previous studies. The integration of a risk scoring mechanism further improves operational efficiency by enabling security teams to focus on the most critical threats first.

VIII. SECURITY AND PRACTICAL IMPLICATIONS

The proposed framework offers a proactive approach to securing web applications by automating the identification and prioritization of exploit vectors. Organizations can incorporate this system into ongoing security monitoring workflows, reducing reliance on manual analysis and enabling faster response to potential threats.

By leveraging the risk scoring mechanism, security teams can concentrate on vulnerabilities with the highest potential impact, enhancing overall management of exploit vectors. Furthermore, the system can facilitate compliance checks and support enforcement of security policies within enterprise environments.

IX. LIMITATIONS

The evaluation of the proposed framework was conducted using a curated experimental dataset and simulated adversarial payloads. While the results demonstrate promising detection performance, several limitations must be considered for real-world deployment.

First, the system has been tested under controlled conditions, and performance may vary when applied to live network environments with high traffic volumes, encrypted communications, or complex web application behaviors. Handling encrypted or obfuscated payloads remains a challenge, as it may limit the system's ability to accurately extract features for classification.

Second, adversarial payload patterns continuously evolve, leading to potential concept drift over time. Without periodic retraining and model updates, detection accuracy may degrade when confronted with novel attack techniques. This requires establishing a robust model maintenance and update strategy to sustain long-term effectiveness.

Third, the computational overhead of ensemble-based classifiers, particularly Random Forest, may impact real-time deployment in resource-constrained environments. High-throughput web applications may require optimized implementations or lightweight model alternatives to ensure minimal latency.

Finally, the risk scoring mechanism, while useful for prioritization, relies on accurate severity estimation, which may vary depending on organizational context or threat landscape. False prioritization could lead to suboptimal allocation of security resources, necessitating careful tuning and validation of the scoring parameters.

Overall, while the proposed framework provides a strong foundation for automated exploit detection, addressing these limitations is essential to ensure reliable and scalable real-world application.

X. CONCLUSION AND FUTURE WORK

In this study, we proposed a predictive framework for detecting application-layer web vulnerabilities by analyzing HTTP traffic patterns. The approach integrates supervised machine learning models with a risk scoring mechanism to automate both the detection and prioritization of potential security threats.

The experimental results demonstrate that the framework achieves robust detection performance while keeping false positives at a minimum. Moreover, the inclusion of the risk scoring component allows security teams to make informed decisions by highlighting vulnerabilities with higher potential impact and confidence.

For future work, we plan to investigate the application of deep learning techniques to enhance detection of complex attack patterns. Expanding the framework to handle encrypted traffic and deploying it in cloud-based architectures are also key objectives. Additionally, we aim to incorporate adaptive learning strategies to accommodate evolving adversarial payloads and mitigate the effects of concept drift in dynamic real-world environments.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2021.

[2] J. Fonseca and M. Vieira, "Web Application Security Testing: A Systematic Literature Review," *IEEE Access*, vol. 5, pp. 10713–10735, 2017.

[3] H. Holm, "Performance of Automated exploit vector Scanning Tools," in *Proc. IEEE Int. Conf. on Availability, Reliability and Security*, 2015.

[4] R. Sommer and V. Paxson, "Outside the Closed World: On Using predictive model for Network Intrusion identification," in *Proc. IEEE Symp. on Security and Privacy*, 2010.

[5] Z. Li, J. Zhang, and X. Wang, "Deep Learning-Based exploit vector identification: A Survey," *IEEE Access*, vol. 9, pp. 14396–14415, 2021.

[6] J. Chen, X. Zhang, and Y. Li, "predictive model for Zero-Day exploit vector identification," *Computers & Security*, vol. 93, 2020.

[7] W. Xiong, R. Lagutin, and J. He, "Cybersecurity Risk Assessment Using predictive model," *Future Generation Computer Systems*, vol. 98, pp. 208–219, 2019.

[8] M. Almorsy, J. Grundy, and I. Müller, "An Analysis of the Cloud Computing Security Problem," in *Proc. Asia-Pacific Software Engineering Conf.*, 2010.

[9] S. Behl, *Cybersecurity and Cyberwar: What Everyone Needs to Know*, Oxford University Press, 2017.

[10] M. Kempanna, P. Pushpa, and M. Rangaswamy, "BERT Model Based Identification and Classification of Web Vulnerabilities Using Deep Learning," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 21s, pp. 236–248, 2024.

[11] A. Sharma, R. Gupta, and S. Verma, "Defect-Scanner: A Comparative Study of Language Models for Software exploit vector identification," *International Journal of Information Security*, vol. 23, pp. 3513–3526, 2024.

[12] A. Alshammari, M. Khan, and R. Kumar, "AI-Driven Anomaly identification system for Cybersecurity in Smart Systems," *Electronics*, vol. 14, no. 12, 2025.